

# Dart Game Features and Implementation Logic

---

## 1. BP\_Weapon\_Projectile\_Base (Projectile Base Class)

### 1.1 Overview

#### Purpose

BP\_Weapon\_Projectile\_Base inherits from BP\_Weapon\_Base and is the base class for all weapons that use a ProjectileMovementComponent (such as darts). It uniformly manages the projectile lifecycle: stationary on spawn → activated on launch → stopped on hit → destroyed after a delay. Concrete weapon subclasses like the dart only need to inherit it and override the Shoot and Drop Weapon events.

#### Description

- BeginPlay: disables the ProjectileMovementComponent on spawn so the weapon stays stationary until picked up/thrown
- OnHitTarget: after hitting a target, stops movement, disables collision, and destroys after a 3-second delay

### 1.2 Lifecycle Management

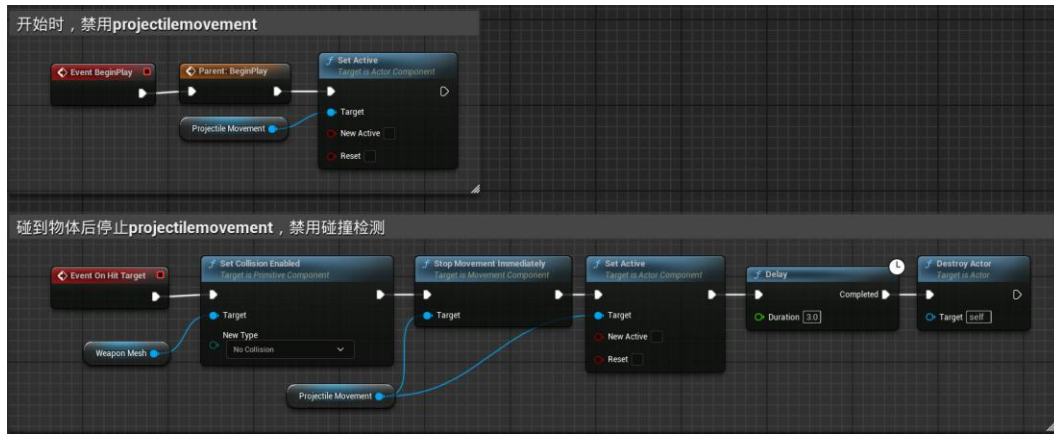
#### Purpose

Manages the projectile's full lifecycle from spawn to destruction, activating/deactivating the movement component at the right times so collision events are not re-triggered after a hit.

#### Execution flow

- BeginPlay (disable movement on spawn)
- OnHitTarget (stop and destroy after a hit)

#### Blueprint screenshots



### 1.3 ProjectileMovementComponent Configuration

#### Purpose

Configures the projectile movement component's physical parameters so the dart's orientation automatically follows its flight direction and standard gravity produces a natural downward arc.

#### Configuration parameters

| Parameter                       | Value     | Description                                                                                                                   |
|---------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------|
| Initial Speed                   | 0.0       | Initial speed is 0; the subclass's Shoot event sets the velocity manually                                                     |
| Max Speed                       | 0.0       | No maximum speed cap                                                                                                          |
| Rotation Follows Velocity       | Checked   | The dart's orientation automatically follows the flight velocity direction (the tip always points along the flight direction) |
| Rotation Remains Vertical       | Unchecked | Lets the dart tilt freely in flight, matching a real dart's arc                                                               |
| Initial Velocity in Local Space | Checked   | The initial velocity is interpreted in local space                                                                            |
| Projectile Gravity Scale        | 1.0       | Standard gravity (100%), producing a natural downward arc                                                                     |

#### Blueprint screenshots

| ▼ Projectile                    |                                     |
|---------------------------------|-------------------------------------|
| Initial Speed                   | 0.0                                 |
| Max Speed                       | 0.0                                 |
| Rotation Follows Velocity       | <input checked="" type="checkbox"/> |
| Rotation Remains Vertical       | <input type="checkbox"/>            |
| Initial Velocity in Local Space | <input checked="" type="checkbox"/> |
| Projectile Gravity Scale        | 1.0                                 |

## Design

- Rotation Follows Velocity = true: the dart rotates automatically in flight, with the tip always facing the current velocity direction, visually pointing at the target like a real dart
- Projectile Gravity Scale = 1.0: applies standard gravity so the dart's trajectory is a natural parabola — the faster the throw the flatter the arc, the slower the throw the faster it drops — matching real physics

## 2. Dart Launch Logic (BP\_Weapon\_Dart Subclass)

### 2.1 Overview

#### Purpose

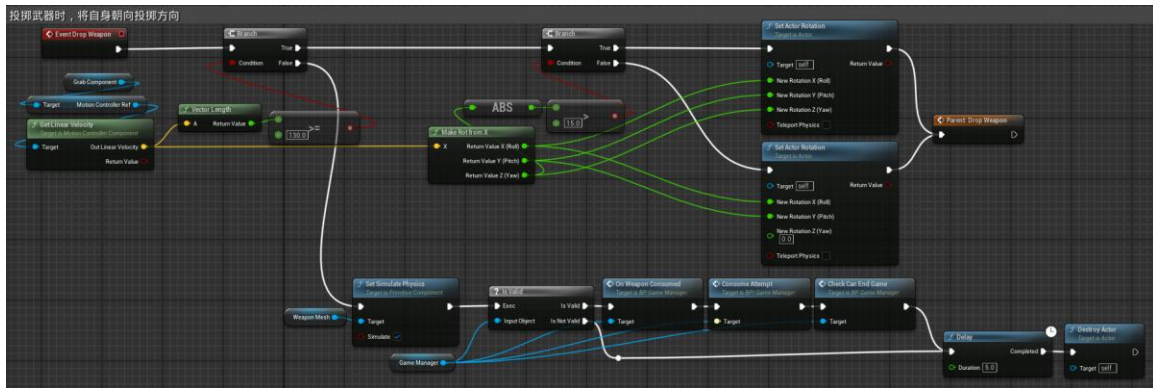
BP\_Weapon\_Dart inherits from BP\_Weapon\_Projectile\_Base and implements the dart-specific throw orientation alignment and launch speed calculation. The core idea: on throw, align the dart's orientation with the controller's velocity direction; on launch, compute the world-space velocity as the dart's own Forward Vector multiplied by a speed scalar.

### 2.2 Drop Weapon Orientation Alignment

#### Purpose

In the VR dart game, when the player releases the dart, the controller's velocity determines whether it is a valid throw; if the speed reaches the threshold the dart's orientation is automatically aligned to the throw direction, the GameManager is notified to consume an attempt and check whether the game ends, and the dart is finally destroyed after a delay.

#### Blueprint screenshots



## Key points

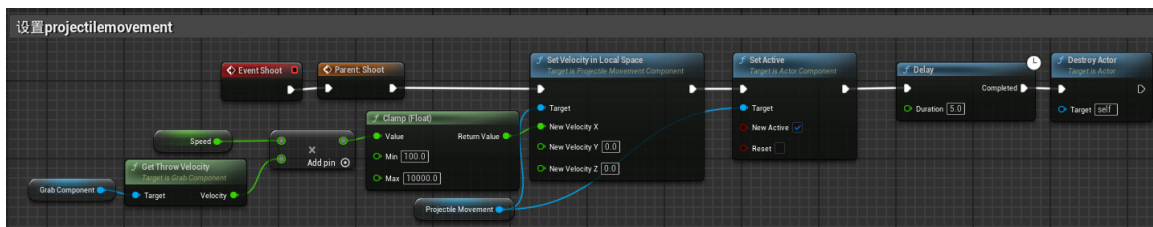
- Throw speed detection: gets the controller's linear velocity and checks whether it is  $\geq 130.0$  as the valid-throw threshold
- Orientation alignment: once the speed qualifies, Make Rot from X (velocity vector) builds a rotation from the velocity direction, correcting the flight orientation and preventing slight controller jitter from producing a bad dart orientation
- Make Rot from X: builds a rotation from the velocity vector with the X axis aligned to the velocity direction, so the dart's Forward Vector (local X axis) points along the throw
- Set the orientation before Parent: Drop Weapon: ensures the dart's orientation is already correct when the parent drop logic runs

## 2.3 Shoot Speed Calculation

### Purpose

When the dart is thrown, the launch speed is computed from the controller's throw velocity and a configured speed factor; Set Velocity in Local Space sets the velocity along the dart's local forward direction, the ProjectileMovementComponent is activated so the dart starts flying, and the dart is automatically destroyed after 5 seconds.

### Blueprint screenshots



## Core principles

- Speed calculation: Get Throw Velocity is called on the Grab Component to get the player's release velocity (scalar)  $\times$  Speed; the direction comes from the dart's own

orientation (already aligned to the throw direction in the Drop Weapon phase), and the magnitude is controlled by the Speed scalar

- The controller velocity direction is not used: controller velocity is only for orientation alignment in the Drop Weapon phase; the Shoot phase uses the dart's own forward direction, ensuring direction consistency
- Set velocity before activating: SET Velocity runs before Set Active so the movement component already has the correct velocity when activated and flight starts immediately

### 3. Dartboard Scoring System (BP\_Target\_Dartboard Subclass)

#### 3.1 Overview

##### Purpose

BP\_Target\_Dartboard inherits from BP\_Target\_Base and is the dart game's scoring target. It computes the score of the dart's hit position through a polar-coordinate transform (angle + distance): the angle determines the sector (1–20 points) and the distance determines the ring (bullseye 50, outer bull 25, triple ring ×3, double ring ×2, miss 0).

##### Description

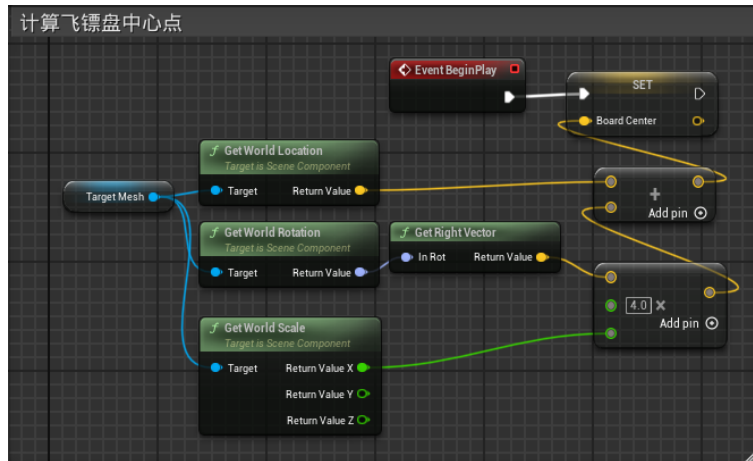
- BeginPlay: computes the dartboard's center point (BoardCenter)
- On Component Hit: detects the dart hit and computes the hit point's angle and distance

#### 3.2 Dartboard Center Calculation

##### Purpose

At game start, computes the dartboard's actual face-center position to serve as the origin for later angle and distance calculations.

##### Blueprint screenshots



## Design

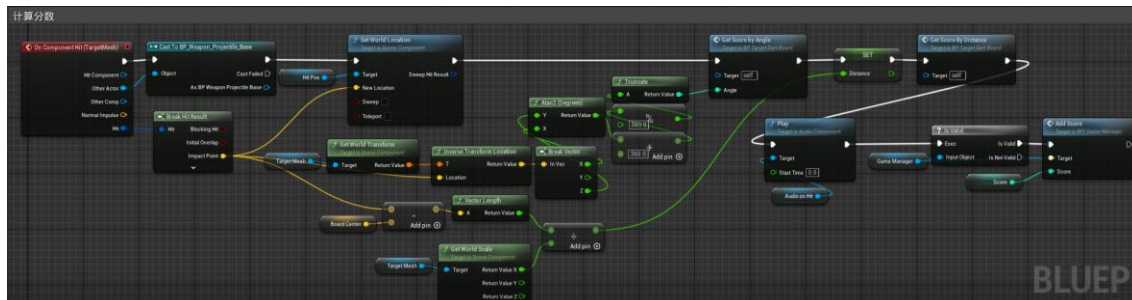
- The dartboard mesh's origin is not at the face center, so it must be offset along Right to the actual face center
- Offset = Right Vector × World Scale: the offset distance scales with the dartboard's scale, growing proportionally when the board is enlarged

## 3.3 Hit-Point Angle and Distance Calculation

### Purpose

When the dart hits the board, the hit point's world coordinates are converted to the dartboard's local coordinates, then polar coordinates (Atan2 for the angle, Vector Length for the distance) determine the sector and ring of the hit position.

### Blueprint screenshots



### Core principles

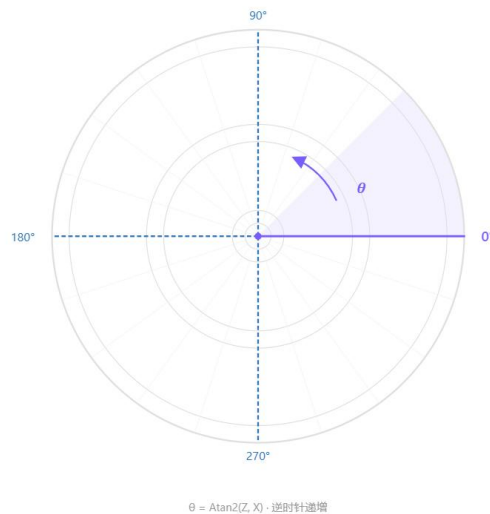
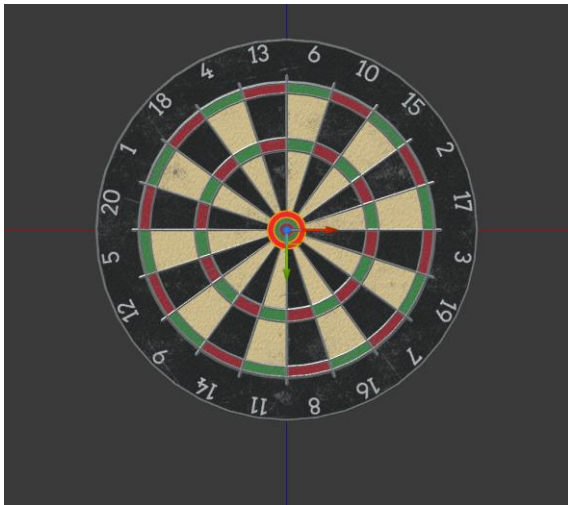
- Inverse Transform Location: converts the hit point from world to dartboard-local coordinates. No matter how the board is rotated or moved in the world, the angle and distance are always computed in the board's own coordinate system, guaranteeing correct results

- $\text{Atan2}(Z, X)$ : the dartboard plane is the XZ plane (normal along Y);  $0^\circ$  is toward positive X (the board's right), the angle increases counter-clockwise, and the range is  $0-360^\circ$
- $\text{Vector Length}(X, Z) \div \text{World Scale}$ : computes the distance in the XZ plane and divides it by the dartboard's scale, normalizing the distance to a standard size so boards of different sizes can share the same score thresholds

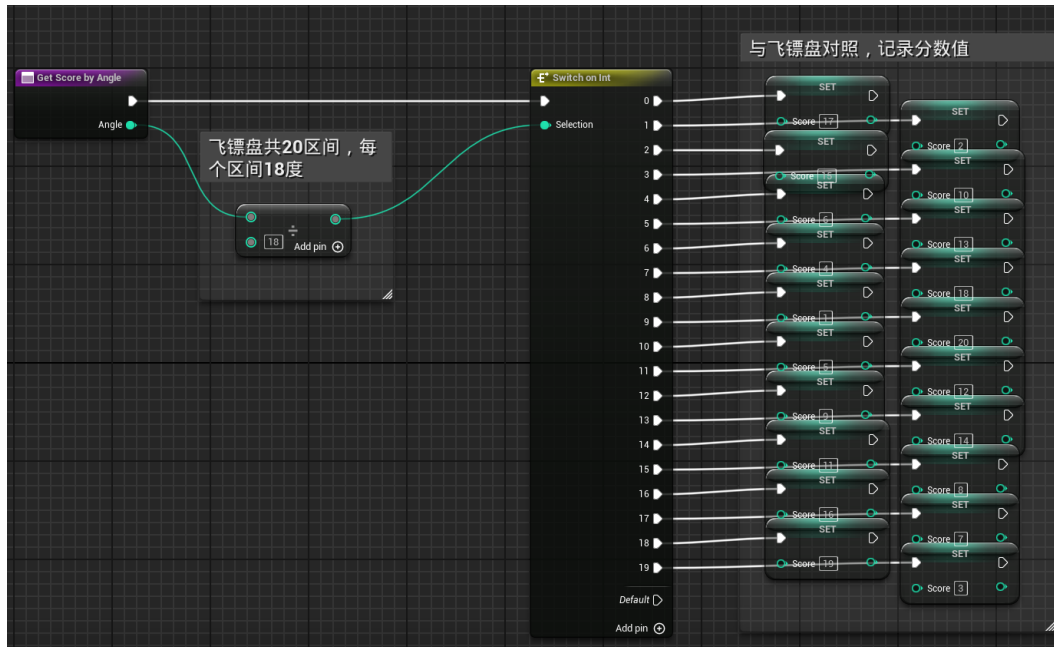
### 3.4 Get Score by Angle (Angle $\rightarrow$ Sector Score)

#### Purpose

Determines the sector the dart hit from the angle ( $0-360^\circ$ ) and returns the corresponding base score ( $1-20$ ). The dartboard has 20 sectors, each spanning  $18^\circ$  ( $360^\circ \div 20 = 18^\circ$ ).



#### Blueprint screenshots



## Core principles

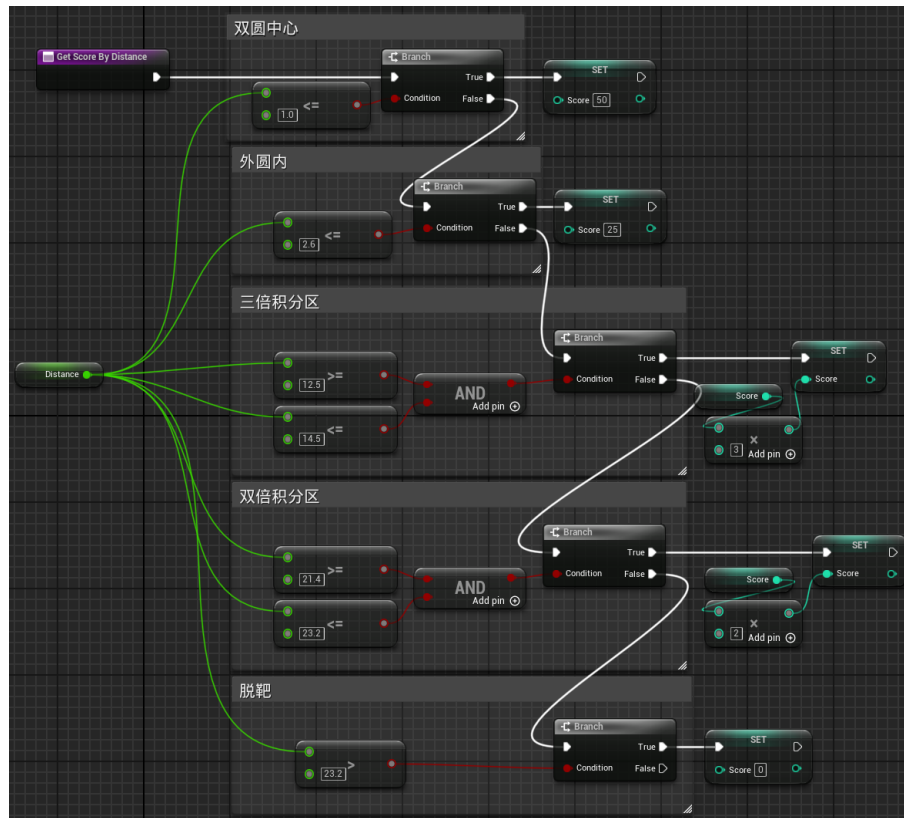
- 20-sector design: a standard dartboard has 20 sectors of 18° each;  $\text{angle} \div 18$  gives an integer index 0–19 corresponding to the 20 scoring regions
- Score values follow the actual dartboard layout: 0° starts at the board's right, increases counter-clockwise, and each index's score matches the number printed on the board

## 3.5 Get Score by Distance (Distance → Ring Multiplier)

### Purpose

Determines the ring type of the hit position from the normalized distance and corrects the base score. The bullseye (50) and outer bull (25) directly override the score; the triple ring ( $\times 3$ ) and double ring ( $\times 2$ ) multiply the base score; the single ring leaves it unchanged; a miss sets it to 0.

### Blueprint screenshots



## Design

- Cascading priority: special regions (bullseye, outer bull) are checked first, then multiplier regions (triple, double), and finally the miss. The priority ensures a bullseye hit directly awards 50 and is never overridden by the triple ring
- Distance thresholds are measured from the actual dartboard: by dragging an object in the scene to each region's real position and reading the normalized distance, the thresholds (1.0, 2.6, 12.5, 14.5, 21.4, 23.2) were determined

## 4. Complete Throwing Flow

### Execution flow

The complete flow from dart spawn to score calculation:

1. The dart spawns in the scene; BeginPlay disables the ProjectileMovementComponent and the dart stays stationary
2. The player's VR controller approaches the dart, the Grab Component triggers a grab, and the dart attaches to the controller
3. The player makes a throwing motion and releases the trigger, firing the Drop Weapon event
4. The controller's linear velocity is read and the dart's orientation is aligned to the velocity direction

5. Parent: Drop Weapon is called to run the parent drop logic
6. The Shoot event fires; Get Actor Forward Vector  $\times$  Speed computes the world-space velocity
7. SET Velocity sets the speed, Set Active(true) activates the movement component, and the dart starts flying
8. Rotation Follows Velocity keeps the dart's tip automatically facing the flight direction
9. Projectile Gravity Scale = 1.0 makes the dart affected by gravity, producing a natural downward arc
10. The dart hits the dartboard, firing On Component Hit
11. Cast To BP\_Weapon\_Projectile\_Base verifies the hitting object is a projectile
12. Break Hit Result gets the ImpactPoint's world coordinates, and the angle and distance are computed
13. Get Score by Angle gets the sector's base score from the angle (1–20 points)
14. Get Score by Distance corrects the score by distance (bullseye/triple/double/miss)
15. Play Sound plays the hit sound and Add Score(GameManager) adds the score to the total
16. BP\_Weapon\_Projectile\_Base's OnHitTarget fires: disable collision  $\rightarrow$  stop movement  $\rightarrow$  3-second delay  $\rightarrow$  destroy